

DISPENSA DELLA LEZIONE

Cicli, incremento e scope

Ripetere istruzioni con while, do / while e for, gestire i contatori e capire dove vivono le variabili.

CAPITOLO 1

Il ciclo while

NOTA

Un ciclo è una struttura che esegue più volte un blocco di istruzioni finché una condizione rimane vera. Quando la condizione diventa falsa, il ciclo si ferma e il programma prosegue con le istruzioni successive.

In JavaScript esistono tre tipi di ciclo: `while`, `do / while` e `for`. I primi due sono adatti quando non si sa in anticipo quante volte il blocco dovrà essere ripetuto. Nel ciclo `while` la condizione viene controllata prima di ogni iterazione: se è falsa fin dall'inizio, il blocco non viene mai eseguito.

Dentro il blocco è fondamentale che qualcosa cambi ad ogni giro, altrimenti la condizione resterebbe sempre vera e il ciclo non terminerebbe mai. Di solito si aggiorna una variabile numerica o un valore booleano.

Esempio

Un programma deve cercare il primo numero intero maggiore di `100` divisibile per `7`. Non sappiamo quante iterazioni servono, sappiamo solo la condizione di uscita.

```
let numero = 101;

while (numero % 7 !== 0) {
  numero = numero + 1;
}

console.log("Primo numero divisibile per 7 dopo 100: " + numero);
```

CAPITOLO 2

Il ciclo `do / while`

NOTA

Il ciclo `do / while` esegue prima il blocco e controlla la condizione solo dopo. Questo garantisce che il blocco venga eseguito almeno una volta, indipendentemente dalla condizione.

La differenza rispetto a `while` riguarda il momento del controllo. Con `while` la condizione viene verificata prima ancora di entrare nel blocco: se è falsa, il blocco non parte mai. Con `do / while` il blocco parte sempre, e solo alla fine si decide se ripeterlo. Questo lo rende la scelta giusta quando la prima esecuzione deve avvenire comunque. Un esempio classico è l'acquisizione dei dati da parte dell'utente: la richiesta di input deve essere mostrata almeno una volta; successivamente, l'algoritmo verifica se il dato è valido e decide se reiterare la domanda, finché l'utente non inserisce un valore valido.

Il blocco si apre con `do` e la condizione si scrive alla fine, dopo la parentesi graffa di chiusura, seguita da punto e virgola.

Esempio

Un programma chiede all'utente di inserire un numero tra `1` e `10`. La richiesta viene ripetuta finché il valore non rientra nell'intervallo valido. La domanda deve comparire almeno una volta, quindi `do / while` è la scelta più adatta.

```
let numero;

do {
  numero = Number(prompt("Inserisci un numero tra 1 e 10:"));
} while (numero < 1 || numero > 10);

console.log("Hai inserito: " + numero);
```

CAPITOLO 3

Il ciclo for

NOTA

Il ciclo **for** è un ciclo controllato: si usa quando si sa già quante volte il blocco deve essere ripetuto. Raccoglie inizializzazione, condizione e aggiornamento in un'unica riga, rendendo il codice più compatto e leggibile.

La struttura è composta da tre parti scritte tra parentesi e separate da punto e virgola. La prima parte dichiara e inizializza la variabile di controllo e viene eseguita una sola volta all'inizio. La seconda parte è la condizione, che viene verificata prima di ogni iterazione. La terza parte aggiorna la variabile al termine di ogni iterazione.

La variabile dichiarata nella prima parte del **for** con **let** esiste solo all'interno del ciclo. Fuori dal blocco non è accessibile.

Per l'aggiornamento si usano spesso le scorciatoie **i++** e **i--**, equivalenti a **i = i + 1** e **i = i - 1**. Quando **++** è scritto prima della variabile (**++i**), il valore viene incrementato prima di essere usato nell'espressione; quando è scritto dopo (**i++**), viene prima usato il valore attuale e poi incrementato.

Esempio

```
let ingressi = 0;

console.log(ingressi);    // 0
console.log(ingressi++);  // 0 - viene usato prima, poi incrementato
console.log(ingressi);    // 1
console.log(++ingressi);  // 2 - viene incrementato prima, poi usato
console.log(ingressi);    // 2
```

Esempio

Stampa i numeri da **1** a **10** usando un ciclo **for**.

```
for (let i = 1; i <= 10; i++) {  
  console.log(i);  
}
```

CAPITOLO 4

Scope delle variabili

NOTA

Lo scope è la zona del codice in cui una variabile è visibile e utilizzabile. Con `let` e `const` lo scope è di blocco: la variabile esiste solo dentro le parentesi graffe `{ }` in cui è stata dichiarata.

Ogni volta che si apre un blocco con `{`, si entra in uno scope più interno. Le variabili dichiarate in quello scope esistono solo al suo interno e non sono visibili fuori. Le variabili dichiarate fuori, invece, sono visibili anche all'interno dei blocchi annidati. Ogni blocco può leggere ciò che sta nel livello superiore, ma il contrario non vale.

Lo scope protegge le variabili da interferenze accidentali. Se una variabile serve solo dentro un ciclo, ha senso dichiararla lì, così non entra in conflitto con il resto del programma. Uno degli errori più comuni è dichiarare una variabile dentro un blocco e poi cercare di usarla fuori: in quel punto la variabile non esiste e il programma va in errore.

Esempio

Un programma calcola il quadrato di tre numeri inseriti dall'utente. La variabile `limite` è dichiarata fuori ed è accessibile ovunque. La variabile `numero` è dichiarata dentro il ciclo e non serve fuori.

```
const limite = 3;

for (let i = 1; i <= limite; i++) {
  const numero = Number(prompt("Inserisci il numero " + i + ":"));
  console.log(numero + " al quadrato è " + numero * numero);
}

console.log("Numeri richiesti: " + limite);
```